SIMATS ENGINEERING

Department of Artificial Intelligence & Machine Learning

ITA05 – COMPUTER VISION

ASSIGNMENT 5 – CO5

Develop, Compare and Justify an OpenCV-Based Recognition and Analysis System for a Smart University Campus Gate

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.

Total Marks: 50

Duration: 1 week

STUDENT DETAILS

Particulars	Details	Particulars	Details
Student Name	M. Sahasra	Register Number	192425317
Department / Section	Artificial Intelligece & Machine Learning	Academic Year / Semester	2025-2026
Faculty Name	Dr. Bindu G	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	_____	Course Code	ITA05

Assignment Title: Develop, Compare and Justify an OpenCV-Based Recognition and Analysis System for a Smart University Campus Gate

MARK SHEET (Rubric-based Evaluation)

Task	Topic	Marks	Marks Awarded
1	Image and Video Reading	4	
2	Image Processing	4	
3	Face Detection	5	
4	Face Recognition	5	
5	Expression Recognition	4	
6	OCR Techniques	4	
7	Motion Analysis	4	
8	Video Recognition	6	
9	Application Integration	6	
10	Application Adaptation	8	
TOTAL		50	

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Notes
1	Agreement between input approaches, feature/strategy comparison and justified video-reading method; measured FPS, frame age, dropped frames and CPU table; installation and 80 ms delay analysis.		
2	Level-annotated original/processed images, Otsu/adaptive/CLAHE comparison, parameter analysis, OCR/foreground quality and processing-time results for at least 10 day/night/glare images.		
3	Face-detection outputs, labelled test plots, precision/recall, false positives/image and timing for Haar, HOG-SVM and DNN; scale/ROI decision.		
4	Face-recognition results for 5,000 identities, FAR/FRR or ROC curves, unknown-person rejection, matching-time/model-size comparison and privacy/consent analysis.		
5	Expression-classifier outputs, three confusion matrices, overall/per-class metrics, inference time, cross-dataset evaluation and anonymity/ethics discussion.		
6	Rectified plate samples, Tesseract/contour-classifier/deep-OCR comparison, character and full-plate accuracy, processing time and low-confidence verification rule.		

7	Corridor motion masks and annotated alert outputs, delay/FPS/false-alarms-per-hour comparison, learning-rate analysis and 12-hour false-alarm calculation.		
8	Annotated gate-video frames, face identity and track results, recognition accuracy, duplicate attendance, ID switches, latency and FPS comparison.		
9	Integrated application and architecture diagram, three pipeline comparison, 40 ms budget, design decision table, data-retention plan and edge/cloud analysis.		
10	12-camera adaptation tests, 180 fps workload calculation, night/IR performance, strategy comparison, 3-second alert analysis, consent-withdrawal deletion evidence and drift-monitoring plan.		

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Stage	Evidence to retain	Used in tasks
Image/video acquisition	Original still images, RTSP/webcam/video samples, camera settings, timestamps, resolution and frame-age logs.	1, 2, 3, 6, 7, 8, 9, 10
Image preprocessing	BGR/gray/RGB conversions, Otsu/adaptive/CLAHE outputs, plate crops and perspective-corrected images.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Face detection	Haar/HOG/DNN/YuNet detections, bounding boxes, confidence values and labelled test results.	3, 4, 5, 8, 9, 10
Face recognition	Enrolment samples/templates, identity-match scores, unknown decisions, FAR/FRR and ROC records.	4, 8, 9, 10
Expression classification	Anonymous face crops, happy/neutral/unhappy predictions, confusion matrices and aggregate metrics.	5, 9
Plate localization and OCR	Plate polygons, rectified plate images, OCR strings, character confidence and verification logs.	2, 6, 9, 10
Motion analysis	Frame differences, MOG2 masks, optical-flow vectors, ROI events and intrusion alert timestamps.	7, 8, 9, 10
Video tracking and recognition	Track IDs, periodic detections, identity votes, annotated frames, ID switches and attendance decisions.	8, 9, 10
Integration and decision	Pipeline block diagram, bounded-queue records, CPU/FPS/latency measurements, alerts and decision table.	1–10
Privacy, retention and drift	Consent/withdrawal records, deletion logs, anonymous expression aggregates, retention checks and drift indicators.	4, 5, 9, 10

MANDATORY SUBMISSION EVIDENCE CHECK

- ☐ All ten tasks are answered in the given order and all three candidate techniques are compared.
- ☐ All reported measurements are obtained by executing the submitted implementation; predictions are clearly separated from observed results.
- ☐ Use identical labelled test images/videos, hardware and parameter settings for fair comparisons.
- ☐ Include commented Python/OpenCV code, screenshots, output images/videos and tables for each task.
- ☐ Report accuracy, precision, recall, FAR, FRR, OCR character/plate accuracy, FPS, latency and false alarms using defined formulas.
- ☐ Respect the 40 ms frame interval for 25 fps and the stated 1-second, 2-second and 3-second response requirements.
- ☐ Keep face data only for the stated 30-day period in the original scenario; expression data must be anonymous.
- ☐ Do not use facial-expression labels as proof of a person's actual emotional state or as the sole basis for consequential decisions.
- ☐ Document consent, transparency, access control, encryption, deletion and withdrawal procedures for personal data.
- ☐ Clearly identify measured, calculated, estimated and illustrative values; do not claim unexecuted tests.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Stage	Input	Output / Evidence
1. Input	Still images, RTSP streams, webcam	Validated BGR frames, timestamps, frame-age/FPS logs
2. Enhancement	Plate/corridor images	Otsu/adaptive/CLAHE outputs and quality comparison
3. Face detection	1080p gate frames	Face boxes, precision/recall and timing
4. Face recognition	Detected face crops + enrolment gallery	Identity or unknown result, FAR/FRR/ROC

5. Expression analysis	Anonymous kiosk face crop	Happy/neutral/unhappy class with uncertainty
6. Plate OCR	Localized/rectified plate crop	Plate string, character confidence and verification
7. Corridor monitoring	Night video	Foreground/motion mask, ROI event and alert log
8. Video recognition	Gate video	Tracked face identity, attendance decision and ID-switch log
9. Integration	All selected modules	Architecture, decision table, CPU/latency measurements
10. Adaptation	12 night/IR camera streams	Revised low-resource design, drift monitoring and compliance

COMPUTER VISION

ASSIGNMENT 5 – CO5

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

SCENARIO AND OPERATING DATA

Sunrise University requires a low-cost OpenCV application for gate face recognition, vehicle number-plate OCR, an anonymous expression-feedback kiosk and after-10-pm restricted-corridor intrusion monitoring. The application must work on still images and live video. A missed or incorrect identity, plate read or intrusion alert must be handled conservatively.

Sunrise University wants to modernize security and administration at its main gate with a low-cost computer-vision application developed using OpenCV. The application must perform recognition – (1) recognize registered students and staff by face, (2) read vehicle number plates using OCR, (3) estimate visitor satisfaction from facial expressions at an anonymous kiosk – and analysis – (4) automatically monitor a restricted laboratory corridor after 10 pm on video and raise an intrusion alert. Recognition and analysis must work on both still images and live video.

You are the vision engineer. In every task, three candidate techniques are given. You must develop all three with OpenCV, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

Parameter	**Value**
Cameras	1920×1080 IP cameras streaming at 25 fps (RTSP); 2 gate lanes, 1 barrier camera, 1 kiosk webcam, 1 corridor camera

Registered people	5,000 students and staff, with 5 enrolment photos each (taken in an office with uniform lighting)
Gate traffic	Up to 30 people per minute per lane; recognition result needed within 1 second of a person reaching the gate line
Vehicles	Plates (about 500 mm wide) read at the barrier with vehicles moving at 10 km/h; day, night and headlight glare
Corridor monitoring	After 10 pm only; lights sometimes switch on/off; a tree moves outside the corridor window; alert within 2 seconds
Processing hardware	One edge PC: 4 CPU cores, 16 GB RAM, no GPU (budget limit); Python + OpenCV
Frame time budget	At 25 fps, one frame arrives every 40 ms
Data policy	Face images and videos retained for at most 30 days; expression data must be anonymous

EXPERIMENTAL SETUP AND METRIC DEFINITIONS

Metric	**Definition**
FPS	Processed frames divided by elapsed seconds.
Frame age	Processing time minus capture timestamp; indicates freshness.
Latency	Time from event/frame capture to final decision or alert.
Precision	TP/(TP+FP); proportion of predicted positives that are correct.
Recall	TP/(TP+FN); proportion of ground-truth positives detected.
FAR	False acceptances / impostor attempts; measure at each recognition threshold.

FRR	False rejections / genuine attempts; measure at each recognition threshold.
OCR character accuracy	Correctly recognized characters / total ground-truth characters.
Plate accuracy	Exact full-plate matches / total plate images.
False alarms/hour	Distinct false alert events divided by monitored hours.
CPU utilisation	Average CPU use during the same fixed-duration test.
Frame interval	1000/25 = 40 ms at 25 fps.

TASK 1 – Develop the OpenCV Input Stage: Compare Image and Video Reading Strategies (4 Marks)

Question / Required Work

Develop image/video input using OpenCV. Compare (A) blocking sequential VideoCapture/read, (B) threaded latest-frame buffer of size one, and (C) grab/retrieve sampling. Include video-file and RTSP/webcam reading, frame age, dropped frames, CPU, 40 ms budget, 80 ms processing delay, installation choice, imread/imdecode, BGR/RGB, hardware decode and batching.

Develop · using OpenCV · analysis on images and videos (installation, reading images and videos, working with videos).The application must read still images (enrolment photos, plate images) and 1080p video streams at 25 fps. Compare three ways of reading video frames.

Technique	**Basic idea and key OpenCV functions**
A. Blocking sequential read	cv2.VideoCapture(url) with cap.read() inside the main loop; each frame is processed before the next is read.
B. Threaded latest-frame reader	A separate thread continuously reads frames into a queue of size 1 so the main loop always gets the newest frame.
C. Grab/retrieve sampling	cap.grab() on every frame but cap.retrieve() (decode) only on every k-th frame to save processing.

- Implement three video-processing approaches for reading and processing frames from a video file and an RTSP/webcam stream: (1) sequential capture and processing, (2) threaded capture with a latest-frame buffer, and (3) threaded capture with a frame queue. Submit the commented Python code and ensure that all reported results are obtained by executing your implementation.
- Compare the three approaches based on frame age (the time elapsed since capture), number of dropped frames, CPU utilization, and their ability to meet the 40 ms per-frame processing budget required for 25 FPS.

- Justify with experimental evidence: Measure FPS, frame age, dropped frames, and CPU utilization for each approach under normal operating conditions and when an artificial processing delay of 80 ms per frame is introduced. Record the results in a comparison table and identify which approach provides the most suitable performance for real-time applications.
- Justify the installation configuration: Select and justify the appropriate OpenCV package (opencv-python, opencv-contrib-python, or a headless variant), specify a compatible pinned version, and configure an isolated Python virtual environment for CPU-only execution. Demonstrate the correct loading of still images using cv2.imread() and cv2.imdecode(), and explain the difference between BGR and RGB colour representations.
- Analyse the failure scenario: Determine what happens when Approach 1 uses an 80 ms processing step despite the 40 ms frame budget. Calculate the expected processing rate and explain how frame age can increase over one minute when frames accumulate in a queue. Distinguish between accumulated processing delay and the age of the newest available frame, and define an appropriate policy for processing, skipping, or discarding stale frames.
- Investigate modern video pipelines: Explain how hardware-accelerated video decoding through GStreamer or FFmpeg can reduce decoding overhead and how batching frames for neural network inference can improve throughput. Discuss the trade-offs between batching, processing latency, memory consumption, and frame freshness, and justify whether these techniques change the selection of the most appropriate video-processing approach for real-time applications.

Evidence Expected: *A comparison table of the three techniques with measured FPS/delay values, a justified choice and setup, and quantified consequences.*

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Blocking sequential read	cv2.VideoCapture(url) with cap.read() inside the main loop; each frame is processed before the next is read.
B. Threaded latest-frame reader	A separate thread continuously reads frames into a queue of size 1 so the main loop always gets the newest frame.

C. Grab/retrieve sampling	cap.grab() on every frame but cap.retrieve() (decode) only on every k-th frame to save processing.

Answer

1.1 Installation and environment

Use a virtual environment so project dependencies do not conflict. For GUI display use opencv-python; for additional modules such as cv2.face use opencv-contrib-python instead of installing both packages together. On a headless server use the corresponding headless package. Pin the tested version in requirements.txt and record Python/OpenCV versions.

```
python -m venv .venv
# Windows
.venv\\Scripts\\activate
python -m pip install --upgrade pip
pip install opencv-contrib-python==4.10.0.84 numpy==1.26.4
python -c "import cv2; print(cv2.__version__)"
```

cv2.imread(path) reads an image from a file and returns a NumPy array in BGR order. If the image bytes are already in memory, cv2.imdecode(np.frombuffer(data, np.uint8), cv2.IMREAD_COLOR) decodes them. If imread returns None, the path, permissions or file format must be checked. Convert BGR to RGB only when required by a plotting or ML library using cv2.cvtColor(frame, cv2.COLOR_BGR2RGB).

```
import cv2
import numpy as np

img = cv2.imread('plate.jpg')
if img is None:
    raise FileNotFoundError('Could not read plate.jpg')
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
raw = open('plate.jpg', 'rb').read()
decoded = cv2.imdecode(np.frombuffer(raw, np.uint8), cv2.IMREAD_COLOR)
```

1.2 Three input strategies

Approach A reads and processes each frame in one loop. It is easy to debug and preserves sequential order, but processing delay blocks the next read and can create stale output. Approach B has a capture thread that continually replaces a one-frame buffer. It favours the newest frame and bounds queue memory, but drops intermediate frames when processing is slower than capture. Approach C grabs incoming frames and decodes only selected frames; it saves decode/processing work but intentionally reduces temporal sampling and can miss short events.

Approach	Advantages	Limitations	Best use
A. Sequential	Simple, ordered, easy to debug.	Read loop stalls when processing is slow; frame age/latency grows.	Offline files and workloads that keep up with source FPS.
B. Latest-frame thread	Freshest frame, bounded memory, avoids long stale queues.	Drops frames; needs thread-safe buffer and clean shutdown.	Live monitoring where freshness is more important than every frame.
C. Grab/retrieve sampling	Can skip expensive decode on unwanted frames.	Lower effective sample rate; brief events may be missed.	Low-priority streams or scheduled sampling.

1.3 80 ms delay calculation

At 25 fps, the camera produces one frame every 40 ms. If processing takes 80 ms/frame, the maximum processing rate is 1000/80 = 12.5 fps. The input rate is 25 fps, so backlog grows at 12.5 frames per second if every frame is queued. Over one minute, 750 frames arrive and only 750? At 12.5 fps, 750 frames are processed; 1500 frames arrive, leaving 750 queued. The oldest queued frame is approximately 30 seconds old (750×40 ms), assuming no queue limit and steady rates. This is accumulated queue delay. A latest-frame buffer instead discards stale intermediate frames; it may process at 12.5 fps while each processed frame is comparatively fresh, subject to capture/decode and scheduling delays.

Use a bounded queue and explicit stale-frame policy: for live security, overwrite/discard older frames and process the newest frame; for offline evidence processing, retain every frame and accept slower-than-real-time completion. Log capture timestamps and dropped-frame counts so skipped frames are visible.

1.4 Modern pipeline considerations

GStreamer or FFmpeg can use hardware-supported decoding where available, reducing CPU load; on the stated no-GPU PC the benefit depends on codec support and whether the CPU has a supported media engine. Batching improves throughput for neural inference by amortizing overhead, but increases waiting time, memory use and frame age. For a 1-second gate decision, large batches are usually undesirable. Neither batching nor hardware decoding removes the need for a freshness-aware input policy.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Sequential	Preserves order and every frame when fast enough.	At 80 ms it reaches only 12.5 fps and queues stale frames.	Offline processing or sufficiently fast pipelines.
Latest-frame thread	Fresh output with bounded memory.	Drops frames; requires synchronization.	Preferred live stream strategy.
Grab/retrieve sampling	Reduces decode workload.	May miss brief events and lowers sample rate.	Secondary low-priority stream sampling.

Evidence to Record and Decision

Measure actual FPS, frame age, dropped frames and CPU with normal processing and with an 80 ms artificial delay. Include the same source clip and machine for all three. Do not substitute the calculated 12.5 fps for a measured benchmark.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 2 – Develop Image Analysis: Compare Image Processing Techniques (4 Marks)

Question / Required Work

Implement global Otsu, adaptive Gaussian thresholding and CLAHE followed by thresholding for plate and corridor images. Evaluate legibility, uneven lighting, noise, block size, clip limit and time; test at least 10 day/night/glare images; annotate with rectangle/putText/polylines; preserve the original frame; analyze headlight-glare failure.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Global Otsu thresholding	Gray conversion, Gaussian blur, then cv2.threshold(..., THRESH_BINARY + THRESH_OTSU) using one threshold for the whole image.
B. Adaptive Gaussian thresholding	cv2.adaptiveThreshold with a Gaussian-weighted local window so the threshold varies across the image.
C. CLAHE followed by thresholding	cv2.createCLAHE for local contrast enhancement, then Otsu or adaptive thresholding.

- Develop using Python/OpenCV: Implement all three image enhancement techniques for license plate and corridor images, including commented code and results obtained from execution.
- Evaluate character legibility under uneven lighting, noise amplification, sensitivity to block size and clip limit, and processing time per image.
- Test at least 10 day, night, and glare images, reporting OCR character accuracy or foreground–background quality and processing time.
- Use cv2.rectangle(), cv2.putText(), and cv2.polylines() to annotate license plate detections, recognised text, and corridor tripwires.
- Demonstrate why annotations must be drawn on a copy of the original frame to preserve unmodified evidence and avoid affecting recognition.

- Analyse the rejected technique under nighttime headlight glare and determine its impact on image quality and character recognition.

Evidence Expected: *Side-by-side output images, a table of accuracy and time for the three techniques, a justified choice and a reasoned order of operations.*

Answer

2.1 Processing techniques

Global Otsu first converts to grayscale and usually applies a small Gaussian blur. Otsu selects one threshold from the image histogram by maximizing between-class variance. It works well when foreground and background form two reasonably distinct intensity groups, but one threshold cannot handle strong illumination variation across the image.

Adaptive Gaussian thresholding computes a local threshold for each neighbourhood using a Gaussian-weighted sum minus a constant C. It is more suitable when one side of a plate is brighter than the other, but small block sizes can amplify noise and large block sizes behave more like global thresholding.

CLAHE divides the image into tiles and enhances local contrast with a clip limit to restrict noise amplification. Apply it to grayscale or the luminance channel, then use Otsu or adaptive thresholding. A high clip limit can exaggerate sensor noise and headlight glare.

```
import cv2


def preprocess(gray):
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    _, otsu = cv2.threshold(blur, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    adaptive = cv2.adaptiveThreshold(gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 31, 7)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)
    _, clahe_otsu = cv2.threshold(enhanced, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return otsu, adaptive, clahe_otsu
```

2.2 Parameter selection and annotation

Test adaptive block sizes such as 21, 31 and 41 (odd values greater than one) and constants C such as 3, 7 and 11. Test CLAHE clip limits such as 1.5, 2.0 and 3.0. Select settings on a validation subset and keep the final test images separate. Record OCR character accuracy or foreground quality and processing time for every image.

Draw annotations on a copy of the original image. This preserves unmodified evidence and prevents drawn rectangles/text from entering the OCR or recognition pipeline. Use cv2.rectangle for boxes, cv2.putText for recognised text and cv2.polylines for a plate polygon or corridor tripwire.

```
display = frame.copy()
cv2.rectangle(display, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.putText(display, plate_text, (x1, y1-8),
        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,0), 2)
pts = np.array([[(100, 300), (900, 300), (900, 340), (100, 340)]], dtype=np.int32)
cv2.polylines(display, pts, True, (255, 0, 0), 2)
```

2.3 Night glare and rejected technique

Global Otsu is likely to fail when bright headlight regions dominate the histogram: the selected global threshold may wash out plate characters or merge them with the plate background. Adaptive thresholding can improve local unevenness but may turn glare edges/noise into false character strokes. CLAHE can recover local contrast in dark regions, but excessive enhancement can amplify noise. The rejected method should be identified from the measured test set, not assumed in advance.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Global Otsu	Fast and simple; good for bimodal, evenly lit images.	Single threshold fails under spatially uneven light and glare.	Clean daylight plate images.
Adaptive Gaussian	Local thresholds handle illumination gradients.	Sensitive to block size/C; can amplify noise.	Unevenly lit plate/corridor regions.

CLAHE + threshold	Improves local contrast before binarization.	High clip limit amplifies noise and glare artifacts.	Low-contrast night frames after tuning.

Evidence to Record and Decision

Use at least 10 images covering day, night and glare. Provide side-by-side original and processed images, per-image OCR/foreground quality, average processing time and selected parameters.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 3 – Develop Feature Detection: Compare Face Detection Techniques (5 Marks)

Question / Required Work

Implement Haar cascade, HOG+linear SVM and DNN SSD face detectors. Compare precision/recall, false positives, full-1080p and downscaled speed, pose/mask/glasses/low-light/small-face robustness; choose scale/ROI; analyze fastest-but-inaccurate and accurate-but-slow choices; compare YuNet or RetinaFace.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Haar cascade	Haar-like rectangle features with a boosted cascade; cv2.CascadeClassifier and detectMultiScale.
B. HOG + linear SVM	Histogram of Oriented Gradients features with a linear SVM detector (OpenCV HOGDescriptor, or a HOG-based face detector).
C. DNN (SSD) detector	Learned convolutional features; cv2.dnn.readNet with blobFromImage and a pre-trained SSD face model.

- Develop (using OpenCV): implement all three techniques in Python/OpenCV for face detection on a labelled image set and submit the commented code; every result below must come from running your own code.
- Compare on: precision and recall, false positives per image, speed at full 1080p and at a downscaled image, and robustness to side pose, masks, glasses, low light and small faces.
- Justify with evidence: run all three on the same labelled test set (include hard cases) and tabulate precision, recall and milliseconds per image; check the result against the 40 ms budget.
- Justify the detection scale you choose (full image, downscaled image or ROI at the gate line).
- What if not: What if the fastest detector is rejected for accuracy – or the most accurate one is run on every full-resolution frame? Estimate the effect on missed faces and on the frame budget.

- Compare the Haar cascade and the DNN face detector with a current detector (YuNet in OpenCV, or RetinaFace) on masked faces, side views and low light.

Evidence Expected: *A comparison table with measured precision, recall and timing for all three detectors, and a defended choice linked to the frame budget.*

Answer

3.1 Detection techniques

Haar cascade uses Haar-like intensity contrast features, integral images and a boosted cascade. It is fast on frontal faces and CPU-friendly, but often misses side views, masks and low-light faces. HOG describes local gradient directions and a linear SVM classifies the descriptor; it captures shape/edge structure but can be slow at full resolution and is sensitive to pose/occlusion. DNN SSD learns multi-level convolutional features and can generalize better to appearance variation, but model inference may exceed the 40 ms per-frame budget on a CPU.

3.2 Evaluation and operating scale

Use a labelled set with face boxes and hard cases. A true positive requires IoU above a defined threshold (commonly 0.5). Precision=TP/(TP+FP), recall=TP/(TP+FN), false positives/image=FP/number of images. Time each method on full 1920×1080 frames and on a downscaled image. Downscaling reduces computation but can remove small face detail. A gate-line ROI is efficient if the physical entry region is fixed; the ROI must still cover every valid approach path.

3.3 What-if analysis

If the fastest Haar detector has lower recall, it may miss genuine faces and should not be selected solely because it is fast. If the most accurate DNN runs on every full-resolution frame and takes more than 40 ms, the input queue can grow and decisions become stale. A practical compromise is ROI detection, resize for candidate discovery, then crop-and-refine at higher resolution for face recognition.

3.4 Current detector

YuNet is a compact face detector available through OpenCV Zoo and is a useful CPU-oriented comparison. RetinaFace is a strong detector with landmark outputs but can be more computationally demanding. Compare them on the same masked, side-view and low-light images; report the actual model version, input size and CPU timing.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Haar cascade	Very fast on frontal, well-lit faces.	Weak on side pose, masks, low light and small faces.	Low-cost baseline / proposal detector.
HOG + SVM	Shape-based features; no large deep model required.	May be slower at 1080p; less robust to pose and occlusion.	Classical comparison baseline.
DNN SSD	Learned features; generally stronger variation handling.	CPU latency and model dependency; may miss tiny faces.	Primary candidate if ROI/scale meets timing.
YuNet/RetinaFace	Modern landmark/face detection comparison.	Runtime depends on model size and CPU.	External current-detector benchmark.

Evidence to Record and Decision

Report precision, recall, FP/image and ms/image for all detectors on the same hard-case test set. Compare each time to 40 ms. State whether the chosen method is full-frame, downscaled or ROI-based.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 4 – Develop Face Recognition: Compare Recognizers (5 Marks)

Question / Required Work

Implement Eigenfaces, LBPH and deep embeddings for 5,000 identities with five enrolment images each. Evaluate accuracy, FAR/FRR/ROC, threshold, unknown rejection, masks/glasses/lighting, enrolment effort, gallery scaling to 50,000, pretrained ArcFace/SFace, and DPDP Act 2023 considerations.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Eigenfaces	PCA-based holistic features; cv2.face.EigenFaceRecognizer_create (opencv-contrib).
B. LBPH	Local Binary Pattern Histograms; cv2.face.LBPHFaceRecognizer_create (opencv-contrib).
C. Deep embedding	Pre-trained network gives a face embedding (e.g., OpenCV FaceRecognizerSF or a dnn model); matching by cosine/L2 distance.

- Implement all three face recognition techniques for student enrolment and test-image recognition, and submit commented code with results obtained from execution.
- Evaluate recognition accuracy under illumination, pose, and expression changes; FAR/FRR at different thresholds; enrolment effort; new-student addition; matching time for 5,000 people; and model size.
- Test enrolment-style and gate-style face images, plot FAR/FRR or ROC curves for all three techniques, justify the gate threshold by balancing security and queue delay, and demonstrate rejection of unknown individuals.
- Analyse the rejected technique when people wear masks or glasses or when gate lighting differs from enrolment conditions, and explain the impact on recognition reliability.
- Evaluate how increasing the gallery from 5,000 to 50,000 people affects matching time, memory requirements, and false acceptance risk.

- Compare LBPH and Eigenfaces with pretrained ArcFace or OpenCV SFace embeddings in terms of accuracy, robustness, computational cost, and suitability for gate-based recognition.
- Explain the consent, transparency, security, retention, and data-subject rights considerations under India's Digital Personal Data Protection Act, 2023, when collecting and storing students' facial data.

Evidence Expected: *FAR/FRR or ROC curves and accuracy/timing tables for the three recognizers, a justified threshold and quantified consequences.*

Answer

4.1 Recognition methods

Eigenfaces applies PCA to aligned grayscale face images and represents each face using a low-dimensional projection. It is simple but holistic: changes in illumination, pose and expression can strongly affect the representation. LBPH computes local binary pattern histograms and compares them using a distance metric. It is relatively lightweight and can tolerate some local texture changes, but large pose/lighting differences and occlusion remain difficult. Deep embedding models map a face crop to a feature vector; identity matching uses cosine similarity or L2 distance. Pretrained ArcFace/SFace embeddings are generally more robust but require careful threshold calibration and model governance.

4.2 Gallery and matching

With 5,000 identities and five enrolment photos each, the gallery contains 25,000 images if all samples are retained. A template-based design can store one or several normalized embeddings per identity, reducing storage and matching work. A new enrolment should include identity verification, consent/notice, quality checks and duplicate detection. Unknown rejection is essential: if the best match is farther than the validated threshold, return UNKNOWN rather than forcing the nearest identity.

4.3 FAR, FRR and threshold

FAR is the proportion of impostor comparisons incorrectly accepted; FRR is the proportion of genuine comparisons incorrectly rejected. Plot both against the similarity/distance threshold or show an ROC curve. A stricter threshold reduces false acceptance but may increase false rejection and gate delay. Select the operating point using security risk, genuine-user queue time and validation data from gate-like images, not only office enrolment photos.

4.4 Gallery scaling and privacy

Growing from 5,000 to 50,000 identities increases the number of candidate templates by a factor of ten if template count per person remains constant. A brute-force embedding comparison has approximately linear search work in gallery size; memory also grows roughly linearly. An index can reduce average search time but must be evaluated for recall and false matches. Masks/glasses hide discriminative regions, and a lighting mismatch changes image statistics; re-capture or a secure secondary verification method may be needed.

Under India's Digital Personal Data Protection Act, 2023, the university should provide clear notice of purpose, obtain valid consent where required, minimize collection, restrict access, secure data, define retention/deletion, support withdrawal and applicable data-subject rights, and document processors and incident handling. This is an academic design discussion, not legal advice. Face templates remain sensitive personal data in practical risk terms even when raw images are deleted.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Eigenfaces	Compact PCA representation and simple implementation.	Sensitive to lighting, alignment, pose and expression.	Educational baseline; not preferred for a large diverse gate gallery.
LBPH	Lightweight local texture histogram; OpenCV contrib support.	Limited robustness to strong pose/illumination changes and masks.	CPU baseline and small controlled deployments.

Deep embedding (ArcFace/SFace)	Discriminative vector representation; scalable similarity search.	Model and threshold governance; compute and privacy risk.	Preferred technical candidate after consent, validation and safeguards.

Evidence to Record and Decision

Provide ROC/FAR-FRR plots, unknown-person rejection examples, matching time for 5,000 identities, model/template size and a scaling estimate for 50,000. Include gate-style and enrolment-style conditions.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 5 – Develop Facial Expression Recognition: Compare Classifiers (4 Marks)

Question / Required Work

Implement handcrafted LBP/HOG+SVM, small CNN from scratch and pretrained/transfer model for happy/neutral/unhappy kiosk classes. Compare per-class accuracy, confusion matrices, posed vs spontaneous generalization, time, class imbalance, privacy, anonymity, bias and limits of emotion inference.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Handcrafted features + SVM	LBP or HOG features from the normalized face crop classified by an SVM (cv2.ml.SVM_create).
B. Small CNN trained from scratch	A compact CNN trained on a public expression dataset subset (e.g., FER2013 or CK+).
C. Pre-trained model via transfer	A pre-trained network (ONNX through cv2.dnn, or fine-tuned) used for expression classification.

- Implement all three expression classification techniques on a labelled facial image dataset and submit commented code with results obtained by executing the program.
- Compare: Evaluate overall and per-class accuracy, confusion between similar expressions, class imbalance, performance on posed versus spontaneous expressions, inference time, and training data requirements.
- Justify with evidence: Generate a confusion matrix for each technique and evaluate generalisation using a second dataset or consent-based, self-collected images.
- Assess reliability and privacy: Determine whether the results are reliable enough for practical use, explain how participant anonymity is protected, and justify why predictions should support rather than replace human judgement.
- Analyse the limitations of applying a model trained on posed expressions to candid kiosk images and the risks of triggering actions automatically without human review.

- Ethical considerations: Examine concerns about demographic bias and the weak relationship between visible facial expressions and actual emotions, and recommend appropriate restrictions on emotion recognition in campus environments.

Evidence Expected: *Confusion matrices and accuracy/timing comparison of three techniques, a cross-dataset check and a reasoned position on reliability.*

Answer

5.1 Techniques

Handcrafted LBP/HOG features describe texture or edge orientation from a normalized face crop and feed an SVM. This approach is compact and can be inspected, but may not capture subtle variation. A small CNN learns features from labelled expression images; it requires more training data and careful augmentation and may overfit a small dataset. A pretrained model transferred to the kiosk classes can improve feature learning, but its training population and label definitions may not match the campus setting.

5.2 Evaluation

Use a held-out test set and confusion matrix for each classifier. Report overall accuracy and per-class precision/recall/F1. If one class dominates, accuracy can be misleading; use macro-F1 and balanced accuracy. Compare posed expressions with spontaneous kiosk images or a second dataset. Record inference time and training data requirements.

5.3 Reliability, anonymity and ethics

Visible facial movements do not reliably reveal a person's actual internal emotional state. A classifier may confuse neutral and unhappy expressions, and performance can differ across demographic groups, lighting, age and culture. Therefore the kiosk should be optional, anonymous, non-identifying and used only for low-stakes feedback. Do not link expression predictions to student records, discipline, access decisions or staff evaluation. Do not trigger consequential actions automatically. Provide a non-camera feedback option and delete temporary face crops after processing.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role

LBP/HOG + SVM	Small and CPU-efficient; interpretable handcrafted descriptors.	Limited representation; sensitive to crop quality and spontaneous expression variation.	Lightweight research baseline.
Small CNN	Learns task-specific features from labelled data.	Needs sufficient balanced data; can overfit and be biased.	Useful if dataset and validation are adequate.
Pretrained transfer model	Leverages learned visual representations; often less training data.	Dataset mismatch and inherited bias; still not a reliable emotion detector.	Compare only for optional anonymous feedback, not consequential use.

Evidence to Record and Decision

Include three confusion matrices, per-class metrics, inference time and cross-dataset results. State explicitly that expression labels are not verified emotional states and that use is optional and anonymous.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 6 – Develop Optical Character Recognition: Compare OCR Techniques (4 Marks)

Question / Required Work

Build Tesseract, contour segmentation + k-NN/SVM, and deep OCR (CRNN/EasyOCR/OpenCV TextRecognitionModel). Include perspective correction with getPerspectiveTransform; compare character/plate accuracy under blur, skew, glare, dirt and fonts; explain raw-image OCR risks and verification; compare Tesseract with PaddleOCR/EasyOCR/TrOCR.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Tesseract OCR	Open-source OCR engine (e.g., via pytesseract) applied to the cleaned plate image.
B. Contour segmentation &classifier	cv2.findContours to isolate characters, then k-NN or SVM on character images (cv2.ml).
C. Deep-learning OCR	A CRNN/text-recognition model (e.g., EasyOCR or cv2.dnn TextRecognitionModel) reading the whole plate.

- Develop using Python/OpenCV: Build three number-plate recognition techniques with perspective correction using cv2.getPerspectiveTransform() and submit commented code with results obtained from execution.
- Compare: Examine character-level and plate-level accuracy under blur, skew, glare, dirt, and non-standard fonts, considering training data requirements and processing speed.
- Justify with evidence: Test all three techniques on the same clean, blurred, skewed, and glare-affected plate images, tabulate character accuracy, plate accuracy, and processing time, and identify errors such as 0/O and 8/B.
- What if not: Explain the limitations of applying OCR directly to raw images without plate localization and deskewing, and specify the verification steps required before making a gate-access decision based on a potentially incorrect character.
- Compare OCR engines: Compare Tesseract with a deep learning-based OCR engine such as PaddleOCR, EasyOCR, or TrOCR on number plates and identity cards, considering accuracy, robustness, processing time, and suitability for real-time applications.

Evidence Expected: *An accuracy/timing table for the three OCR techniques with error analysis, a justified choice and a verification rule for low-confidence reads.*

Answer

6.1 Plate rectification

Before OCR, localize the plate and estimate its four corner points in consistent order. Use cv2.getPerspectiveTransform(source,destination) to compute a homography and cv2.warpPerspective to obtain a front-facing plate crop. Convert to grayscale, denoise and threshold using the best Task 2 method. OCR on a raw full image may read background signs, merge characters or interpret skewed text incorrectly.

```
import cv2
import numpy as np

src_pts = np.float32([[x1,y1],[x2,y2],[x3,y3],[x4,y4]])
dst_pts = np.float32([[0,0],[440,0],[440,120],[0,120]])
M = cv2.getPerspectiveTransform(src_pts, dst_pts)
plate = cv2.warpPerspective(frame, M, (440,120))
gray = cv2.cvtColor(plate, cv2.COLOR_BGR2GRAY)
```

6.2 OCR techniques

Tesseract applies a general OCR engine to the cleaned plate crop. It is easy to integrate and can be constrained using a character whitelist and page segmentation mode, but it may confuse similar characters and struggle with blur or unusual fonts. Contour segmentation isolates individual glyphs; each glyph is resized and classified by k-NN or SVM. This is explainable but depends on accurate segmentation and representative character training samples. Deep OCR reads a sequence of characters using learned visual features and sequence modelling; it can handle spacing variation better but needs more compute and validation.

6.3 Verification rule

Do not open a barrier from one low-confidence OCR string. Validate expected plate format, character count and per-character confidence. Require repeated consistent reads across multiple frames, compare with an authorized vehicle list, and request human/security confirmation when reads disagree or confidence is low. Preserve the original image separately from the annotated result. Errors such as 0/O and 8/B should be explicitly included in the test set.

6.4 Engine comparison

PaddleOCR and EasyOCR are deep-learning OCR toolkits; TrOCR is a transformer-based recognizer. Compare them with Tesseract on the same plate and identity-card images. Identity-card OCR contains different layouts and fields, so report it separately from plate accuracy. Model download size and CPU latency matter on the edge PC.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Tesseract	Mature open-source OCR; CPU friendly and configurable.	Sensitive to blur, skew, glare and font variation.	Good baseline after plate rectification.
Contours + k-NN/SVM	Character-level decisions are inspectable.	Segmentation errors propagate; needs labelled glyph dataset.	Controlled font/plate-style baseline.
Deep OCR	Sequence recognition can tolerate spacing and character variation.	Higher compute/model size; confidence calibration needed.	Preferred if measured CPU performance meets gate timing.

Evidence to Record and Decision

Report character accuracy, exact plate accuracy and ms/plate for clean, blurred, skewed, glare and dirty examples. Include error examples and a low-confidence/manual-verification policy.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 7 – Develop Video Analysis: Compare Motion Analysis Techniques (4 Marks)

Question / Required Work

Implement frame differencing, MOG2 and dense Farneback optical flow for corridor ROI/virtual-line alerts. Compare delay, misses, light/shadow false alarms, slow/stationary people, parameter sensitivity and FPS; calculate 12-hour false alarms at one/hour; compare with YOLO+ByteTrack.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Frame differencing	cv2.absdiff between consecutive frames, then threshold, morphology (dilate/erode) and contour area filter.
B. MOG2 background subtraction	cv2.createBackgroundSubtractorMOG2 with shadow detection; adaptive Gaussian-mixture background model.
C. Dense optical flow	cv2.calcOpticalFlowFarneback to estimate motion vectors; regions with large coherent motion are flagged.

- Create three motion detection methods for corridor surveillance and configure ROI-entry or virtual-line-crossing alerts. Submit commented code and output videos generated by executing the program.
- Examine the three methods based on intrusion detection delay, missed detections, false alarms caused by lighting changes and shadows, detection of slow or stationary people, parameter sensitivity, and FPS on an edge PC.
- Test all three methods on identical corridor videos containing normal nighttime activity, sudden lighting changes, moving trees, and slow-moving intruders. Present a comparison table showing detection delay, false alarms per hour, and FPS.
- Determine how high and low background learning rates affect slow-intruder detection and false alarms. Calculate the number of false alarms expected during a 12-hour night shift when the system generates one false alarm per hour.

- Compare with deep learning: Compare frame differencing with YOLO and ByteTrack for corridor people counting, using detection accuracy, counting errors, tracking performance, processing speed, and computational requirements as evaluation criteria.

Evidence Expected: *A table of delay, false-alarm rate and speed for the three techniques, a justified choice with alert rule, and quantified consequences of mis-tuned parameters.*

Answer

7.1 Methods

Frame differencing computes cv2.absdiff between consecutive grayscale frames, thresholds the difference and applies morphology. It reacts quickly to motion but is sensitive to illumination changes and may fail when a person stops. MOG2 maintains an adaptive Gaussian-mixture background model and can mark shadows separately. It handles recurring background variation better but may absorb a slow/stationary intruder if learning is too fast. Farneback dense optical flow estimates a motion vector at each pixel; coherent motion regions can be detected, but it is more computationally expensive and can react to moving trees or lighting changes.

7.2 Alert logic

Define a polygon ROI for the restricted corridor and/or a virtual line. Trigger when a sufficiently large connected component or coherent tracked region enters the ROI and persists for a small number of frames. Exclude the tree region outside the corridor window. Use a short temporal confirmation and merge repeated detections into one incident event. A stationary person may stop producing frame-difference motion, so combine motion onset with object persistence/tracking or a detector.

7.3 Learning rate and false alarms

A high MOG2 learning rate adapts quickly to light changes but may absorb a slow intruder into the background. A low learning rate preserves foreground evidence longer but may produce prolonged false alarms after lighting changes and shadows. At one false alarm per hour over a 12-hour night shift, expected false alarms = $1 \times 12 = 12$. This is an expected count, not a guarantee that exactly 12 occur.

7.4 Deep-learning comparison

YOLO detects people semantically in each selected frame; ByteTrack associates detections across frames to maintain IDs and count people. This is more robust to static background changes than pure frame differencing, but the no-GPU CPU budget may limit FPS. Compare detection/counting accuracy, ID switches, throughput and CPU usage.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Frame differencing	Very low computation and fast response.	Fails on stationary people; light switches create large differences.	Baseline for motion onset.
MOG2	Adaptive background; can model changing background modes.	Learning rate may absorb slow intruders; shadows/tree motion cause false alarms.	Preferred classical background method after ROI and learning-rate tuning.
Farneback flow	Dense motion direction and magnitude.	More CPU intensive; lighting and tree motion can confuse it.	Motion characterization where compute permits.
YOLO + ByteTrack	Semantic people detection and persistent tracks.	Higher CPU/model cost; detector can miss small/occluded people.	Deep-learning benchmark and counting option.

Evidence to Record and Decision

Use identical corridor videos with light switching, tree motion and slow intruders. Report delay, misses, false alarms/hour and FPS. Include the 12-hour false-alarm calculation.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 8 – Develop Recognition on Video: Compare Video Recognition Strategies (6 Marks)

Question / Required Work

Build video face recognition with selected detector/recognizer and compare (A) recognize every frame, (B) detect+track and recognize once per track with periodic re-verification, (C) periodic detection plus multi-frame voting. Compare accuracy, duplicate attendance, ID switches, latency under 1 s, FPS, occlusion/crossing/low light; discuss tracker drift, action recognition and vision-language models.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Recognize every frame	Detect and recognize in every (or every k-th) frame independently; each result is treated as a new decision.
B. Detect + track, recognize once per track	Detect a face, follow it with an OpenCV tracker (cv2.TrackerCSRT_create / KCF), recognize at the first good frame and re-verify periodically.
C. Periodic detection + multi-frame voting	Detect every k frames and combine identity scores over N frames (majority vote or average distance) before deciding.

- Construct three video-based identity recognition approaches using Python/OpenCV for gate surveillance, incorporating face detection, identity matching, tracking, and annotated output videos with bounding boxes and identity labels. Submit commented source code and execution results.
- Compare the approaches in terms of identity recognition accuracy, incorrect or duplicate attendance records, ID switches, decision latency within one second, processing FPS, and robustness to occlusion, intersecting pedestrian paths, and varying illumination.
- Test the approaches using identical video sequences containing single-person entry, people crossing, partial face occlusion, and low-light conditions. Present comparative tables and graphs for recognition accuracy, ID switches, processing latency, and FPS using the same hardware and configuration.

- Examine the computational overhead of processing every full-resolution frame and determine the consequences of incorrect identity assignment caused by tracker drift. Recommend safeguards to prevent unauthorised entry and inaccurate attendance records.
- Investigate the suitability of action-recognition networks and vision–language models for identifying unusual activities in gate surveillance videos. Compare their detection capabilities, computational requirements, inference latency, and feasibility for real-time deployment.

Evidence Expected: *Commented code, annotated output video frames, a measured comparison of the three strategies and a justified choice that meets the 1-second requirement.*

Answer

8.1 Three strategies

Recognize-every-frame independently detects and matches faces each frame. It can correct previous mistakes quickly but repeats expensive detection/recognition and may create duplicate attendance events. Detect-and-track recognizes at the first good frame, tracks the face using CSRT/KCF and re-verifies periodically. This reduces recognition calls but tracker drift can move the box onto another person. Periodic detection with multi-frame voting detects every k frames and combines identity evidence across N frames. It reduces computation and makes decisions more stable, but the vote window adds latency.

8.2 Identity and attendance safeguards

Assign a temporary track ID to each person. Create an attendance event only after a high-confidence identity is confirmed by multiple valid observations and the track crosses the gate line in the permitted direction. Enforce one attendance event per identity per entry session. If two people cross or a face is occluded, mark the track uncertain and re-detect rather than transferring identity. Never use a tracker ID alone as proof of identity.

8.3 Timing and action recognition

The gate requirement is one second from reaching the gate line. Include capture, detection, recognition, voting, logging and any queue delay in the measured decision latency. Processing every full-resolution frame may exceed the CPU budget and produce stale results.

Action-recognition networks and vision-language models can describe unusual activities but are usually heavier than a compact detector/tracker; on the stated CPU they should be considered optional offline or low-frequency analysis unless benchmarks prove real-time feasibility.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Every frame	Fresh recognition each frame; rapid correction possible.	Highest repeated compute; duplicate attendance and flicker risk.	Small ROI/low-rate streams where latency dominates.
Detect + track	Fewer recognizer calls; continuous track ID.	Tracker drift, ID switches and occlusion errors.	Good when track quality is high and re-verification is periodic.
Periodic + voting	Stable multi-frame identity evidence; reduced compute.	Voting window adds delay; may reject short visits.	Preferred compromise if decision remains under 1 second.

Evidence to Record and Decision

Include annotated frames/video, identity accuracy, duplicate attendance count, ID switches, latency and FPS for single entry, crossing paths, occlusion and low-light clips. Document re-verification and unknown-person safeguards.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 9 – Develop the Integrated Application: Compare Integration Architectures (6 Marks)

Question / Required Work

Integrate face recognition, plate OCR, corridor intrusion monitoring and optional expression kiosk. Compare sequential single-thread, event-triggered and multithreaded producer-consumer architectures. Measure FPS, latency, CPU, alerts, recognition accuracy and complexity; provide block diagram, 40 ms budget, design decision table, 30-day data management, two applications, failure scenario and edge-vs-cloud comparison.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Sequential single-thread pipeline	Each frame passes through all modules in order in one loop.
B. Event-triggered pipeline	A cheap motion detector gates the heavy modules; recognition and OCR run only when something moves into the ROI.
C. Multithreaded producer–consumer pipeline	Separate threads/processes for reading, detection, recognition and alerting, connected by queues.

- Develop and Demonstrate: Build an integrated image and video analysis application using Python/OpenCV by combining the three selected techniques from Tasks 1–8. Demonstrate its operation on test images and videos, and submit commented source code with results obtained through execution.
- Compare System Performance: Compare the three approaches using FPS, per-frame latency, CPU utilisation, alert response time, recognition accuracy under challenging conditions, and code complexity.
- Validate Experimentally: Run all three approaches on the same machine using identical images and videos, including low-light, blurred, occluded, and glare-affected cases. Tabulate the results and determine whether the observations support or contradict the predictions made in Tasks 1–8.
- Prepare the System Design: Draw a block diagram covering image/video input, preprocessing, detection, face/plate/expression recognition, decision-making, annotation, alerts, and logging.

Allocate the 40 ms processing budget required for 25 FPS and prepare a Design Decision Table with Decision, Justification (Evidence), and What If Not columns.

- Specify Data Management and Applications: Identify the data stored, its purpose, and its retention period under a 30-day policy. Suggest two additional real-world applications suitable for the proposed integrated architecture.
- Examine Failure Scenarios: Discuss the limitations of testing only with clean daytime images and videos. Identify one experimental result that contradicts the theoretical prediction and explain the possible reasons for the difference.
- Compare Deployment Strategies: Compare on-device edge AI processing with cloud-based vision processing in terms of latency, operational cost, network dependency, privacy, and suitability for real-time security monitoring.

Evidence Expected: *A working, commented application, a measured comparison of three architectures, an original block diagram and decision table, and a defended final design.*

Answer

9.1 Architecture options

A sequential single-thread pipeline sends every frame through all modules in order. It is easiest to debug but the slowest module blocks the rest. An event-triggered pipeline runs a cheap motion/ROI check first and invokes face recognition or OCR only when a person/vehicle enters the relevant region. This saves compute but a poor gate detector can suppress an event. A multithreaded producer-consumer pipeline separates stream capture, detection, recognition, OCR, alerting and logging with bounded queues. It improves utilization but requires synchronization, thread-safe state, queue limits and careful shutdown.

9.2 Proposed block diagram

IP cameras / still images → input readers and timestamps → preprocessing / ROI selection → face detector → face recognizer and unknown rejection; barrier camera → plate localization/rectification → OCR and verification; corridor camera → motion detector / ROI line crossing → intrusion event; anonymous kiosk → optional expression classifier → anonymous aggregate only; all outputs → decision rules → annotation copy → alert service and audit log → retention/deletion controller.

9.3 40 ms budget

At 25 fps, each stream provides a frame every 40 ms. A possible planning allocation is capture/decode 5 ms, resize/ROI 3 ms, primary detector 15 ms, tracking/motion 7 ms, decision/logging 3 ms and spare 7 ms. These are design targets only. If several streams share one CPU, per-stream work must be scheduled and measured; four cores do not provide an independent 40 ms budget to every camera.

Pipeline	Strength	Risk	Decision
Sequential	Simple and easy to validate.	One slow OCR/recognition stage blocks all frames.	Use for offline validation, not all live streams.
Event-triggered	Avoids heavy work on empty scenes.	Missed motion can suppress recognition/OCR.	Use as a soft gate with periodic full checks.
Producer-consumer	Parallel capture and modular workers; bounded queues.	Synchronization and CPU contention; queue design matters.	Preferred integrated architecture with latest-frame queues.

9.4 Design decision table

Decision	Justification (Evidence to provide)	What if not?
Bounded latest-frame queues	Frame-age and CPU measurements under load.	Unbounded queues create stale decisions.
ROI/event-triggered OCR	Plate OCR only at barrier ROI; measured CPU reduction.	Full-frame OCR wastes compute and may read irrelevant text.
Unknown face rejection	FAR/FRR curve and threshold validation.	Forced nearest match can misidentify an unknown visitor.
Human verification for uncertain plate	Character confidence and repeated-read agreement.	Incorrect plate may cause unauthorized barrier action.
Separate anonymous kiosk data	No identity linkage or raw face retention.	Expression output can become personal profiling.

30-day deletion control	Retention job and deletion audit log.	Data remains longer than stated policy.

9.5 Data management and applications

Store only necessary records: identity templates/IDs for authorized gate recognition, event timestamps, camera ID, confidence/decision status and security incident metadata. Raw face images/video should be retained no longer than the stated 30 days and access restricted. Expression data should be anonymous and aggregated, with no identity link. Two additional applications are (1) library entry and occupancy monitoring and (2) parking access/vehicle whitelist verification, subject to consent, purpose limitation and human review.

9.6 Failure scenarios and edge/cloud

Testing only clean daytime images hides low-light, glare, motion blur, occlusion, network jitter and camera-specific variation. A theoretical winner may perform worse because the test data is small, image compression changes detail, or parameter tuning overfits. Edge processing reduces network dependence and sends less personal video off-site, with low local latency; cloud processing can scale compute but adds network latency, bandwidth cost and privacy exposure. For time-critical security events, edge processing is preferred, with carefully controlled remote management rather than continuous cloud video transfer.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role
Sequential single-thread	Simple control flow and debugging.	Blocking stages cause frame delay and poor scaling.	Use as reference baseline.
Event-triggered	Runs expensive modules only when relevant ROI activity occurs.	Gate false negatives can suppress important processing.	Use carefully with periodic safety scans.

Multithreaded producer-consumer	Separates input and modules; supports bounded freshness queues.	Concurrency bugs and CPU contention.	Preferred with explicit queue policies and monitoring.

Evidence to Record and Decision

Submit a working integrated demo, architecture diagram, 40 ms planning budget, decision table, data-retention design and measured comparison on identical challenging samples.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

TASK 10 – Re-develop and Adapt the Application (8 Marks)

Question / Required Work

Adapt to 12 outdoor parking cameras at 1080p/15 fps on the same 4-core CPU/no-GPU PC, poor light/headlight glare/IR grayscale, no face-image storage, consent withdrawal, alerts within 3 s. Compare A frame skipping+ROI, B motion-triggered cascade, C lighter IR-suited techniques. Recompute throughput/budget; assess original methods; propose templates/event logs; drift monitoring; RETAIN/MODIFY/REDESIGN.

Candidate Techniques Given in Assignment

Technique	Basic idea and key OpenCV functions
A. Frame skipping + ROI processing	Process every k-th frame and only regions of interest on all 12 streams; keep the original modules.
B. Motion-triggered cascade	A cheap motion detector (e.g., MOG2 on a small frame) gates the heavy modules on each stream.
C. Lighter / IR-suited OpenCV techniques	Replace heavy or color-dependent modules with lighter techniques that work on infrared gray frames (e.g., CLAHE + HOG people detection, LBPH, classical OCR).

- Develop (using OpenCV): implement all three techniques in Python/OpenCV for the adapted application and test it on night, infrared and multi-stream test videos and submit the commented code; every result below must come from running your own code.
- Impact Analysis : What has changed? Which original assumptions (frame rate, number of streams, lighting, color information, data storage, consent) are now invalid, and why? What if the original application is deployed unchanged?
- Evaluation of the Original Application: Evaluate quantitatively whether your methods still fit the frame-time budget for 12 streams (recompute total frames per second and per-frame time) and whether your detector, recognizer, OCR and motion model still work on infrared/low-light frames. Which parts remain valid?

- Compare and Adapt : Compare strategies A, B and C on throughput, accuracy, alert delay and legal compliance using your own measurements; then describe the revised application (including storing only templates or event logs instead of face images) and what changed.
- Evidence-Based Defence: Defend your final decision with calculations, your Task 9 measurements, hardware and legal limits and the trade-offs accepted. State one condition under which your decision would no longer hold.
- Monitor and Determine: Design a monitoring strategy for the deployed system to detect data drift caused by changes in uniforms, vehicle types, and seasonal lighting. Define measurable drift indicators and performance thresholds to determine when model retraining is required.

Evidence Expected: *Three adaptation strategies developed and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.*

Answer

10.1 Recalculate workload

The original system has several 25 fps streams. The changed requirement is 12 cameras × 15 fps = 180 incoming frames per second. The aggregate arrival interval is 1000/180 ≈ 5.56 ms per frame if frames are interleaved. Each individual camera still produces one frame every 66.67 ms. If the same CPU must process every frame, the average total processing budget is only about 5.56 ms per incoming frame across all streams, far below the original 40 ms single-stream frame interval. The system cannot assume full-resolution processing of every frame by every heavy module.

10.2 Original assumptions that no longer hold

Camera count rises to 12 and frame rate per camera changes to 15 fps. Outdoor night conditions, headlight glare and infrared grayscale invalidate assumptions about stable daylight color. The original face-image retention policy is superseded by the new prohibition on storing face images. Consent can be withdrawn at any time, so enrolment templates and derived records need a withdrawal/deletion workflow. A three-second alert deadline is longer than the original corridor two-second deadline but must be measured end-to-end.

If the original application is deployed unchanged, the CPU will be overloaded, queues will grow, alerts may arrive late, color-dependent preprocessing may fail on IR frames, and retaining face images would violate the new rule. Heavy models on all frames may reduce throughput and make the system operationally unsafe.

10.3 Compare adaptation strategies

Strategy A uses frame skipping and ROI processing on all 12 streams. It limits compute and focuses on relevant areas, but may miss brief events between sampled frames. Strategy B uses a low-resolution MOG2 motion trigger to invoke heavier modules. It saves compute when scenes are quiet but may fail under headlight glare or miss a stationary person. Strategy C replaces heavy/color-dependent modules with lighter IR-compatible methods such as CLAHE on grayscale, HOG people detection and classical OCR. It is CPU-friendly but may have lower recognition accuracy and needs dedicated IR testing.

Strategy	Throughput	Accuracy / delay risk	Compliance
A. Frame skipping + ROI	Predictable reduction in processed frames.	Short events may occur between samples; latency depends on k.	Can avoid storing face images if templates/logs only.
B. Motion-triggered cascade	Efficient when most scenes are static.	Glare/light changes can trigger constantly; stationary events may be missed.	Must still honour consent withdrawal and data deletion.
C. Lighter IR-suited methods	Lower CPU cost; operates on grayscale IR.	HOG/LBPH/classical OCR may be less accurate under poor image quality.	Easier to minimize data, but template handling still needs controls.

10.4 Revised design and privacy

Use bounded latest-frame queues for all cameras. Run a lightweight motion/ROI stage at reduced resolution; schedule the primary detector on selected frames and event regions. Use CLAHE for IR/low-light grayscale where validated, HOG or a compact grayscale-compatible people detector, and plate OCR only in a defined barrier/parking ROI.

Store no face images. If face recognition remains legally and operationally permitted, store only protected templates and minimal event logs, encrypt them, restrict access, support withdrawal by deleting the associated template and linked records, and document deletion completion. A template is still personal data in risk terms; deletion of images alone does not remove privacy obligations.

10.5 Three-second alert and final decision

Use an event-triggered pipeline with a maximum bounded queue age, priority scheduling for active incidents and a local alert output. A three-second target should include capture, detection, confirmation, decision and notification. Use a small number of confirmation frames and avoid waiting for large batches. The recommended decision is REDESIGN: 12 streams and new privacy/IR constraints materially change the workload and data architecture. Retain the useful OpenCV input, ROI, OCR and event-logging components, but replace the original scheduling and storage model.

10.6 Drift monitoring

Monitor measurable indicators: average brightness and contrast, fraction of saturated/headlight pixels, grayscale/IR mode proportion, blur score (e.g., variance of Laplacian), detection confidence distribution, face/plate OCR confidence, unknown/reject rate, false alarms per hour, missed-event rate from periodic audits, and latency/FPS per camera. Trigger review when a metric crosses a predeclared threshold for several consecutive monitoring windows, for example a 20% relative rise in false alarms or 15% drop in recall on audited samples. Retraining is required only after labelled review confirms drift; do not retrain automatically on unverified predictions.

10.7 Condition where decision changes

The redesign assumes the edge PC can sustain the selected sampled workload within the three-second deadline. If measured worst-case latency exceeds three seconds or audited pedestrian/vehicle recall falls below the safety target, the design no longer holds. The university must reduce stream sampling, add edge hardware or distribute processing across additional on-premise devices; it should not silently move identifiable video to a cloud service.

Three-Technique Comparison

Technique	Strengths	Limitations / risks	Recommended role

A. Frame skipping + ROI	Simple, bounded workload reduction.	Can miss brief events and delay detection according to sampling interval.	Useful for low-priority streams with known ROIs.
B. Motion-triggered cascade	Efficient when no activity occurs.	Glare and lighting changes cause false triggers; static people may be missed.	Use as a soft trigger plus periodic detector refresh.
C. Lightweight IR-suited methods	Lower CPU and compatible with grayscale infrared.	Accuracy may decline; each IR camera requires validation.	Use as part of redesigned low-resource pipeline.

Evidence to Record and Decision

Show the 180 fps aggregate calculation, per-camera and aggregate timing, measured throughput, alert latency, IR/night accuracy, consent-withdrawal deletion test and drift indicators. State RETAIN/MODIFY/REDESIGN and one condition that invalidates the choice.

Reporting note: The explanations and calculations are based on the assignment brief and standard OpenCV principles. Replace all benchmark placeholders with results from executing your own code on the required data. Do not report predicted values as measured results.

OVERALL CONCLUSION

The proposed campus-gate application combines OpenCV input handling, image enhancement, face detection and recognition, plate OCR, corridor motion analysis and video-level identity tracking. No single technique is best for every condition. The final design should select methods using measured accuracy, latency, CPU usage and failure behaviour on the same representative test set.

For live streams, freshness-aware bounded queues and ROI-based processing are important. Face recognition must reject unknown or low-confidence matches; plate reads must be verified before access decisions; corridor alerts should combine spatial ROI rules with temporal evidence. Expression analysis should remain optional, anonymous and non-consequential because visible expressions do not reliably establish internal emotion.

Under the changed 12-camera night/IR requirement, the original design should be redesigned around aggregate throughput, low-resource grayscale-compatible processing, bounded queues, three-second alert timing and strict deletion/withdrawal controls. All final choices must be supported by actual execution results.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Image and Video Reading	4
Task 2 – Image Processing	4
Task 3 – Face Detection	5
Task 4 – Face Recognition	5
Task 5 – Expression Recognition	4
Task 6 – OCR Techniques	4
Task 7 – Motion Analysis	4
Task 8 – Video Recognition	6
Task 9 – Application Integration	6
Task 10 – Application Adaptation	8
Total	50